Credit Card Approval Prediction System

Project Description

The **Credit Card Approval Prediction System** is a Machine Learning-based web application developed to predict whether a customer's credit card application is likely to be approved or rejected. Financial institutions receive thousands of credit card applications every day, making manual evaluation a time-consuming process. This project automates the approval process by analyzing applicant details and predicting the approval status using machine learning algorithms.

The system collects applicant information such as gender, annual income, occupation, education level, family status, housing type, age, employment experience, number of children, and family members. These details are preprocessed and passed to a trained Random Forest model, which predicts whether the application is likely to be approved or rejected. The prediction result and confidence score are displayed through a user-friendly Flask web application.

The project demonstrates the complete machine learning lifecycle, including data collection, preprocessing, feature engineering, model training, evaluation, and deployment. It provides an efficient and reliable solution for assisting banks in making faster and more consistent credit card approval decisions.

Social Impact

The Credit Card Approval Prediction System contributes positively to society by improving the efficiency and fairness of the credit card approval process. It helps reduce manual errors and enables financial institutions to make quicker decisions based on applicant information.

The system provides applicants with faster responses regarding their applications, reducing waiting time and improving customer satisfaction. By using historical data and machine learning techniques, the prediction process becomes more consistent and less dependent on manual judgment.

The project also serves as an educational example of how Artificial Intelligence and Machine Learning can be applied in the banking sector to automate decision-making and improve operational efficiency.

Business Impact

Banks and financial institutions process a large number of credit card applications every day. Manual verification of applicant details requires significant time and human effort, which increases operational costs and may lead to inconsistent decisions.

The Credit Card Approval Prediction System helps financial institutions by automating the evaluation process. The system analyzes applicant information and predicts approval status within a few seconds, allowing organizations to process more applications efficiently.

The project helps organizations achieve the following benefits:

- Faster credit card approval process.

- Reduced manual workload.
- Improved consistency in approval decisions.
- Better customer experience.
- Support for data-driven decision making.
- Reduced operational costs.

Existing Solutions

Most banks currently use manual verification processes combined with predefined business rules to evaluate credit card applications. These methods require employees to review applicant information, verify financial details, and make approval decisions.

Although some organizations have partially automated their systems, many still rely heavily on manual verification, which increases processing time and may introduce human errors.

The proposed Credit Card Approval Prediction System improves this process by using Machine Learning techniques to analyze applicant information and predict approval status automatically. The system provides instant predictions while maintaining consistent decision-making based on historical data.

Technology Stack

The project was developed using the following technologies:

Programming Language

- Python 3

Machine Learning

- Scikit-learn
- Random Forest Classifier
- Logistic Regression
- Decision Tree Classifier
- XGBoost
- SMOTE (Synthetic Minority Oversampling Technique)

Web Development

- Flask
- HTML
- CSS

Data Processing

- Pandas
- NumPy

Data Visualization

- Matplotlib
- Seaborn

Model Persistence

- Joblib

Development Tools

- Visual Studio Code
- Git
- GitHub
- Render

References

The following resources were used during the development of this project:

1. Scikit-learn Official Documentation – https://scikit-learn.org/
2. Flask Official Documentation – https://flask.palletsprojects.com/
3. Pandas Documentation – https://pandas.pydata.org/
4. NumPy Documentation – https://numpy.org/
5. XGBoost Documentation – https://xgboost.readthedocs.io/
6. Imbalanced-learn Documentation – https://imbalanced-learn.org/
7. UCI Machine Learning Repository – https://archive.ics.uci.edu/
8. GitHub Documentation – https://docs.github.com/
9. Render Documentation – https://render.com/docs

Project Scenario

Banks receive a large number of credit card applications every day. Each application contains personal, financial, and employment-related information that must be verified before making an approval decision.

Manually reviewing every application requires significant time and effort, especially when the number of applications increases. Manual evaluation may also result in inconsistent decisions due to human errors or varying judgment among employees.

The Credit Card Approval Prediction System addresses this problem by using Machine Learning algorithms to analyze applicant information and predict whether the application is likely to be approved or rejected.

The system performs the following tasks:

- Collects applicant information through a web interface.
- Preprocesses and validates the input data.
- Encodes categorical values.
- Applies the trained Random Forest model.
- Predicts the approval status.
- Displays the prediction result along with the confidence score.

This solution helps financial institutions make faster, more accurate, and consistent approval decisions while reducing manual effort.

System Architecture

The Credit Card Approval Prediction System follows a Machine Learning-based architecture integrated with a Flask web application. The system collects applicant details from the user through a web interface, preprocesses the data, applies the trained machine learning model, and displays the prediction result.

The overall architecture consists of the following components:

- User Interface (HTML & CSS)
- Flask Application
- Data Preprocessing Module
- Feature Engineering Module
- Label Encoding
- Trained Random Forest Model
- Prediction Result Page

The workflow begins when the user enters the required applicant information. The Flask application validates the input, performs preprocessing, and converts categorical values using the saved label encoders. The processed data is then passed to the trained Random Forest model, which predicts whether the credit card application is likely to be approved or rejected. Finally, the prediction result along with the confidence score is displayed to the user.

Prerequisites

Before developing the Credit Card Approval Prediction System, the following software and tools were installed:

Software Requirements

- Python 3.13
- Visual Studio Code
- Git
- GitHub
- Web Browser (Google Chrome)

Python Libraries

The following Python libraries were installed using the requirements.txt file:

- Flask
- Pandas
- NumPy
- Scikit-learn
- XGBoost
- Imbalanced-learn (SMOTE)
- Joblib
- Matplotlib
- Seaborn

Installation command:

pip install -r requirements.txt

```
Flask==3.1.1
gunicorn==23.0.0
pandas==2.3.1
numpy==2.3.1
scikit-learn==1.7.0
xgboost==3.0.2
imbalanced-learn==0.14.0
joblib==1.5.1
matplotlib==3.10.3
seaborn==0.13.2
```

Project Workflow

The Credit Card Approval Prediction System follows the below workflow:

1. Collect the applicant dataset.
2. Clean and preprocess the data.
3. Handle missing values.
4. Perform feature engineering.
5. Encode categorical variables.
6. Split the dataset into training and testing sets.
7. Balance the training dataset using SMOTE.
8. Train multiple machine learning models.
9. Compare model performance.
10. Select the best-performing model.
11. Save the trained model using Joblib.
12. Develop the Flask web application.
13. Accept user input through the web interface.
14. Predict the credit card approval status.
15. Display the prediction result and confidence score.

Milestone 1: Data Collection and Preprocessing

Dataset Collection

The project uses two publicly available datasets:

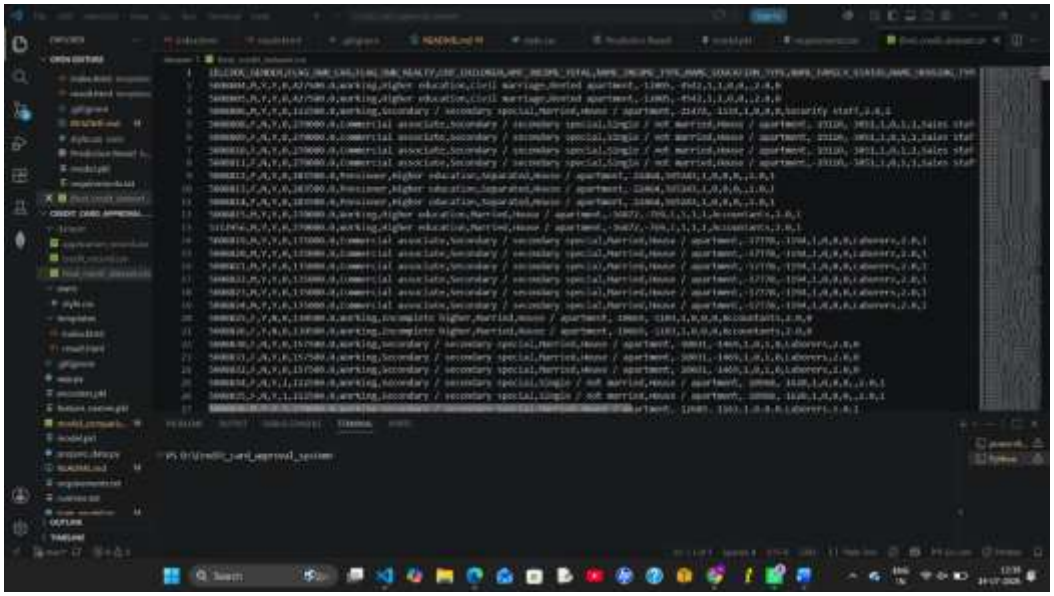
- application_record.csv
- credit_record.csv

The application dataset contains applicant details such as gender, annual income, education level, occupation, housing type, employment details, and family information. The credit record dataset contains historical credit information for each applicant.

Both datasets were merged using the Applicant ID to create a single dataset for model training.

After preprocessing and feature engineering, the final dataset was saved as:

final_credit_dataset.csv



This dataset contains only the required features used for model training.

Data Cleaning

Data cleaning was performed to improve the quality of the dataset before training the machine learning models.

The following preprocessing steps were performed:

- Removed unnecessary columns such as ID.
- Filled missing values in the OCCUPATION_TYPE column using the value "Unknown".
- Removed constant columns such as FLAG_MOBIL.
- Checked for duplicate records.
- Verified the dataset for missing values and inconsistencies.

These preprocessing steps helped improve the reliability of the dataset.

Feature Engineering

Feature engineering was performed to create meaningful features from the existing dataset.

Two new features were created:

AGE

The AGE feature was calculated using the DAYS_BIRTH column.

Formula:

AGE = DAYS_BIRTH / 365

YEARS_EMPLOYED

The YEARS_EMPLOYED feature was calculated from the DAYS_EMPLOYED column.

Positive values were converted to zero because they represented unemployed applicants.

Formula:

YEARS_EMPLOYED = DAYS_EMPLOYED / 365

After creating these new features, the original columns were removed:

- DAYS_BIRTH
- DAYS_EMPLOYED

This improved the readability of the dataset and reduced unnecessary complexity.

Handling Missing Values

The OCCUPATION_TYPE column contained missing values.

Instead of deleting these records, the missing values were replaced with:

Unknown

This allowed the model to retain more training samples while handling missing information effectively.

Label Encoding

Machine learning algorithms require numerical input. Therefore, categorical columns were converted into numerical values using Label Encoding.

The following columns were encoded:

- CODE_GENDER
- FLAG_OWN_CAR
- FLAG_OWN_REALTY
- NAME_INCOME_TYPE
- NAME_EDUCATION_TYPE
- NAME_FAMILY_STATUS
- NAME_HOUSING_TYPE
- OCCUPATION_TYPE

Separate LabelEncoder objects were created and saved using Joblib.

Saved files:

- encoders.pkl
- feature_names.pkl

These files are later loaded in the Flask application to encode user input before prediction.

Train-Test Split

The processed dataset was divided into training and testing datasets.

Training Data : **80%**

Testing Data : **20%**

The dataset was split using the train_test_split() function with stratified sampling to maintain the class distribution in both training and testing datasets.

Applying SMOTE

The original dataset contained an imbalance between approved and rejected applications.

To solve this issue, the Synthetic Minority Oversampling Technique (SMOTE) was applied only to the training dataset.

SMOTE generated additional samples for the minority class, resulting in a balanced dataset.

This improved the model's ability to correctly predict both approved and rejected applications while reducing bias toward the majority class.

Milestone 2: Model Building and Evaluation

After preprocessing the dataset and balancing it using SMOTE, multiple Machine Learning algorithms were trained and evaluated to identify the most suitable model for predicting credit card approval. Different classification algorithms were compared based on their performance, and the best-performing model was selected for deployment.

Logistic Regression Model

Logistic Regression was implemented as the first baseline classification algorithm. It predicts the probability of whether a credit card application is approved or rejected based on the applicant's information.

The Logistic Regression model was trained using the balanced training dataset generated after applying SMOTE. The model provided a simple and interpretable approach for binary classification. Although it achieved satisfactory accuracy, its performance was comparatively lower than the ensemble models.

Decision Tree Model

The Decision Tree Classifier was implemented to learn decision rules from the applicant data.

The model recursively divided the dataset into different branches based on important features such as annual income, occupation type, age, education level, and employment experience. Decision Trees are easy to visualize and interpret, but they are more prone to overfitting when compared to ensemble techniques.

The model produced reasonable prediction accuracy but was outperformed by the Random Forest model.

Random Forest Model

The Random Forest Classifier was selected as the primary machine learning algorithm for this project.

Random Forest is an ensemble learning technique that combines multiple decision trees to improve prediction accuracy and reduce overfitting. Each tree independently predicts the approval status, and the final prediction is determined by majority voting.

The model was trained using the balanced training dataset after applying SMOTE.

The following parameters were used while training the Random Forest model:

- Number of Trees (n_estimators): 50
- Maximum Depth: 10
- Minimum Samples Split: 5
- Minimum Samples Leaf: 2
- Maximum Features: sqrt
- Class Weight: balanced
- Random State: 42

These optimized parameters reduced the model size while maintaining high prediction accuracy.

The trained Random Forest model was selected as the final model and saved using Joblib as:

model.pkl

XGBoost Model

The XGBoost Classifier was also trained and evaluated for comparison.

XGBoost is an advanced boosting algorithm that builds decision trees sequentially to improve prediction performance. It generally provides excellent accuracy for structured datasets.

Although XGBoost achieved competitive performance, the Random Forest model was selected because it provided a good balance between prediction accuracy, training time, model size, and ease of deployment.

Model Comparison

All four machine learning algorithms were trained using the same training dataset and evaluated on the testing dataset.

The following models were compared:

- Logistic Regression
- Decision Tree
- Random Forest
- XGBoost

Each model was evaluated using several performance metrics to determine its effectiveness.

The comparison results were stored in:

model_comparison.csv

The model with the highest overall performance was selected for deployment in the Flask application.

Evaluation Metrics

Several evaluation metrics were used to compare the performance of the machine learning models.

Accuracy

Accuracy represents the percentage of correctly classified credit card applications.

It is calculated as:

Accuracy = Correct Predictions / Total Predictions

A higher accuracy indicates better prediction performance.

Precision

Precision measures how many of the applications predicted as approved were actually approved.

A higher precision value indicates fewer false approvals.

Recall

Recall measures how many of the actual approved applications were correctly identified by the model.

Higher recall ensures that eligible applicants are not incorrectly rejected.

F1-Score

The F1-Score combines Precision and Recall into a single metric.

It provides a balanced measure of model performance, especially when the dataset is imbalanced.

Confusion Matrix

The Confusion Matrix provides a detailed summary of prediction results.

It consists of four components:

- True Positives (Correctly Approved)
- True Negatives (Correctly Rejected)
- False Positives (Incorrectly Approved)
- False Negatives (Incorrectly Rejected)

The confusion matrix helps evaluate the strengths and weaknesses of the classification model.

ROC Curve

The Receiver Operating Characteristic (ROC) Curve was plotted to evaluate the classification performance of the trained model.

The ROC Curve compares:

- True Positive Rate
- False Positive Rate

A curve closer to the upper-left corner indicates better classification performance.

ROC-AUC Score

The Area Under the ROC Curve (ROC-AUC) measures the ability of the model to distinguish between approved and rejected applications.

A higher ROC-AUC score indicates better predictive capability.

Feature Importance

The Random Forest model calculates the importance of each input feature.

Feature Importance analysis helps identify which applicant characteristics have the greatest impact on credit card approval prediction.

The top important features were visualized using a bar chart generated with Matplotlib.

This analysis provides better understanding of the factors influencing the prediction process.

Saving the Best Model

After comparing all machine learning models, the best-performing model was automatically selected.

The selected model was saved using the Joblib library.

The following files were generated:

- model.pkl
- encoders.pkl
- feature_names.pkl
- model_comparison.csv

These files are later loaded by the Flask application to perform real-time predictions without retraining the model.

Project Summary

After completing model training and evaluation, the project generated the following outputs:

- Trained Machine Learning Model (model.pkl)
- Label Encoders (encoders.pkl)
- Feature Names (feature_names.pkl)
- Model Comparison Report (model_comparison.csv)
- Accuracy Comparison Graph
- Feature Importance Graph
- ROC Curve

The Random Forest Classifier demonstrated the best overall performance and was selected as the final prediction model for deployment.

```
# MODELS
# ============================================================


models = {


    "Logistic Regression":
        LogisticRegression(
```

```
        max_iter=2000,
        random_state=42
    ),

"Decision Tree":
    DecisionTreeClassifier(
        random_state=42
    ),

"Random Forest":
RandomForestClassifier(
    n_estimators=50,
    max_depth=10,
    min_samples_split=5,
    min_samples_leaf=2,
    max_features="sqrt",
    class_weight="balanced",
    random_state=42,
    n_jobs=-1
),

"XGBoost":
    XGBClassifier(
        random_state=42,
        eval_metric="logloss",
        use_label_encoder=False
    )

}

# ============================================================
```

```
# TRAIN MODELS
# ============================================================

results = []

best_model = None
best_model_name = ""
best_accuracy = 0

print("\n")
print("="*70)
print("MODEL TRAINING STARTED")
print("="*70)

for name, model in models.items():

    print("\n")
    print("="*60)
    print(name)
    print("="*60)

    # Train Model
    model.fit(X_train, y_train)

    # Prediction
    prediction = model.predict(X_test)

    # Accuracy
    accuracy = accuracy_score(
        y_test,
        prediction
```

```
)

results.append(
    {
        "Model": name,
        "Accuracy": accuracy
    }
)

print("Accuracy :", round(accuracy,4))

print("\nConfusion Matrix")

print(
    confusion_matrix(
        y_test,
        prediction
    )
)

print("\nClassification Report")

print(
    classification_report(
        y_test,
        prediction
    )
)

# Save Best Model
```

```
if accuracy > best_accuracy:

    best_accuracy = accuracy

    best_model = model

    best_model_name = name
```

Perfect! Below is **Part 4** of your documentation.

Milestone 3: Flask Application Development

After training and saving the machine learning model, a Flask web application was developed to allow users to predict credit card approval through a simple and interactive web interface. Flask acts as the bridge between the user interface and the trained machine learning model.

The Flask application loads the trained model, accepts applicant details from the user, preprocesses the input data, predicts the approval status, and displays the result along with the prediction confidence.

Application Architecture

The Flask application consists of the following components:

- HTML User Interface
- Flask Backend
- Trained Random Forest Model
- Label Encoders
- Prediction Module
- Result Page

The application follows the workflow shown below:

User → Flask Application → Data Validation → Label Encoding → Random Forest Model → Prediction → Result Page

---

Loading the Trained Model

The trained machine learning model was saved using the Joblib library after model training. During application startup, the Flask application loads the following files:

- model.pkl
- encoders.pkl
- feature_names.pkl

These files allow the application to make predictions without retraining the model every time the application starts.

User Input Collection

The Home page contains an input form where the applicant enters all the required information.

The application collects the following details:

- Gender
- Own Car
- Own House
- Annual Income
- Number of Children
- Income Type
- Education Type
- Family Status
- Housing Type
- Age
- Years Employed
- Work Phone
- Phone
- Email
- Occupation Type
- Number of Family Members

After entering all the required details, the user clicks the **Predict** button.

Data Validation

The Flask application validates every input received from the HTML form before sending it to the machine learning model.

The validation process ensures that:

- All required fields are entered.

- Numerical fields contain valid numeric values.
- Categorical values match the trained label encoder values.
- Invalid inputs are handled gracefully without crashing the application.

If any invalid input is detected, an error message is displayed to the user.

Label Encoding

The trained machine learning model accepts only numerical values. Therefore, categorical values entered by the user are converted into numerical values using the saved Label Encoders.

The following fields are encoded:

- Gender
- Own Car
- Own House
- Income Type
- Education Type
- Family Status
- Housing Type
- Occupation Type

The encoded values are arranged according to the order stored in **feature_names.pkl** before being passed to the prediction model.

Prediction Process

After preprocessing the input data, the Random Forest model predicts whether the applicant is likely to receive credit card approval.

The prediction process consists of the following steps:

1. Receive applicant information.
2. Validate the input values.
3. Encode categorical features.
4. Arrange the features in the correct order.
5. Pass the data to the trained Random Forest model.
6. Predict the approval status.
7. Calculate the prediction confidence.
8. Display the result.

Prediction Confidence

The Random Forest model supports probability prediction using the predict_proba() function.

The prediction confidence represents the probability of the predicted class.

For example:

- Approved – 92.5%
- Rejected – 87.3%

Displaying the confidence score helps users understand how strongly the model supports its prediction.

Result Generation

After prediction, the Flask application displays:

- Credit Card Approved

or

- Credit Card Rejected

along with the prediction confidence percentage.

The result page also provides an option to return to the home page and perform another prediction.

Milestone 4: Frontend Development

A simple and user-friendly web interface was developed using HTML and CSS.

The interface allows users to enter applicant details and receive prediction results without interacting directly with the machine learning model.

The frontend consists of three main pages:

- Home Page
- Prediction Form
- Result Page

Home Page

The Home Page serves as the entry point of the application.

It provides a clean interface with the project title and a prediction form. Users can easily enter applicant details without requiring technical knowledge.

The page is designed using HTML and CSS to provide a responsive and attractive layout.

Prediction Form

The Prediction Form collects all the information required for predicting credit card approval.

The form contains the following fields:

- Gender
- Own Car
- Own House
- Annual Income
- Number of Children
- Income Type
- Education Type
- Family Status
- Housing Type
- Age
- Years Employed
- Work Phone
- Phone
- Email
- Occupation Type
- Family Members

Input validation ensures that mandatory fields are completed before submission.

This improves data accuracy and prevents invalid predictions.

Result Page

After submitting the form, the applicant is redirected to the Result Page.

The Result Page displays:

- Approval Status (Approved or Rejected)
- Prediction Confidence Percentage

The page also provides a button to return to the Home Page and perform another prediction.

The result page was designed to provide clear and understandable feedback to the user.

User Interface Features

The web application provides the following features:

- Simple and responsive interface.
- Easy-to-use prediction form.
- Fast prediction response.
- Error handling for invalid inputs.
- Displays prediction confidence.
- Attractive result page.
- User-friendly navigation.

Advantages of Flask in this Project

Flask was selected because it is lightweight, flexible, and easy to integrate with machine learning models.

The advantages include:

- Simple project structure.
- Easy integration with Python.
- Fast development.
- Supports HTML templates.
- Easy deployment on cloud platforms.
- Suitable for machine learning web applications.

Milestone 5: Project Deployment

After successfully developing and testing the Credit Card Approval Prediction System, the project was prepared for deployment so that users could access the application through a web browser.

The project source code was uploaded to GitHub for version control and collaboration. The repository contains all the required project files, including the Flask application, machine learning scripts, templates, static files, and project documentation.

The application was deployed using **Render**, a cloud platform that supports Python-based web applications. During deployment, the required Python packages were installed using the requirements.txt file, and the Flask application was configured using Gunicorn.

The deployment process involved the following steps:

1. Upload the project to GitHub.
2. Create a new Web Service on Render.
3. Connect the GitHub repository.

4. Configure the build and start commands.
5. Install all dependencies.
6. Deploy the Flask application.
7. Verify that the application is accessible through the deployed URL.

Deployment allows users to access the system online without installing Python or additional software.

GitHub Repository

GitHub was used for maintaining the project source code and version control.

The repository includes:

- Flask Application
- Machine Learning Scripts
- HTML Templates
- CSS Files
- Requirements File
- README Documentation
- Project Documentation

GitHub made it easier to manage changes, track updates, and collaborate on the project.

Render Deployment

Render was used as the cloud hosting platform to deploy the Flask application.

The deployment configuration included:

Build Command

pip install -r requirements.txt

Start Command

gunicorn app:app

After successful deployment, users can access the application using the generated Render URL.

Application Testing

The Credit Card Approval Prediction System was thoroughly tested to ensure that all functionalities worked correctly.

The following test cases were performed:

Test Case 1: Application Launch

Objective:
Verify that the Flask application starts successfully.

Expected Result:
The Home Page loads without errors.

Status:
Passed

Test Case 2: Valid Applicant Details

Objective:
Verify prediction using valid applicant information.

Expected Result:
The system predicts either "Approved" or "Rejected" along with the confidence score.

Status:
Passed

Test Case 3: Invalid Input

Objective:
Verify the application's ability to handle invalid user input.

Expected Result:
The application displays an appropriate error message without crashing.

Status:
Passed

Test Case 4: Prediction Confidence

Objective:
Verify that the confidence percentage is displayed correctly.

Expected Result:
The prediction confidence appears along with the prediction result.

Status:
Passed

Test Case 5: Navigation

Objective:
Verify navigation between pages.

Expected Result:
Users can return to the Home Page after viewing the prediction result.

Status:
Passed

Performance Testing

The application was tested to ensure smooth performance.

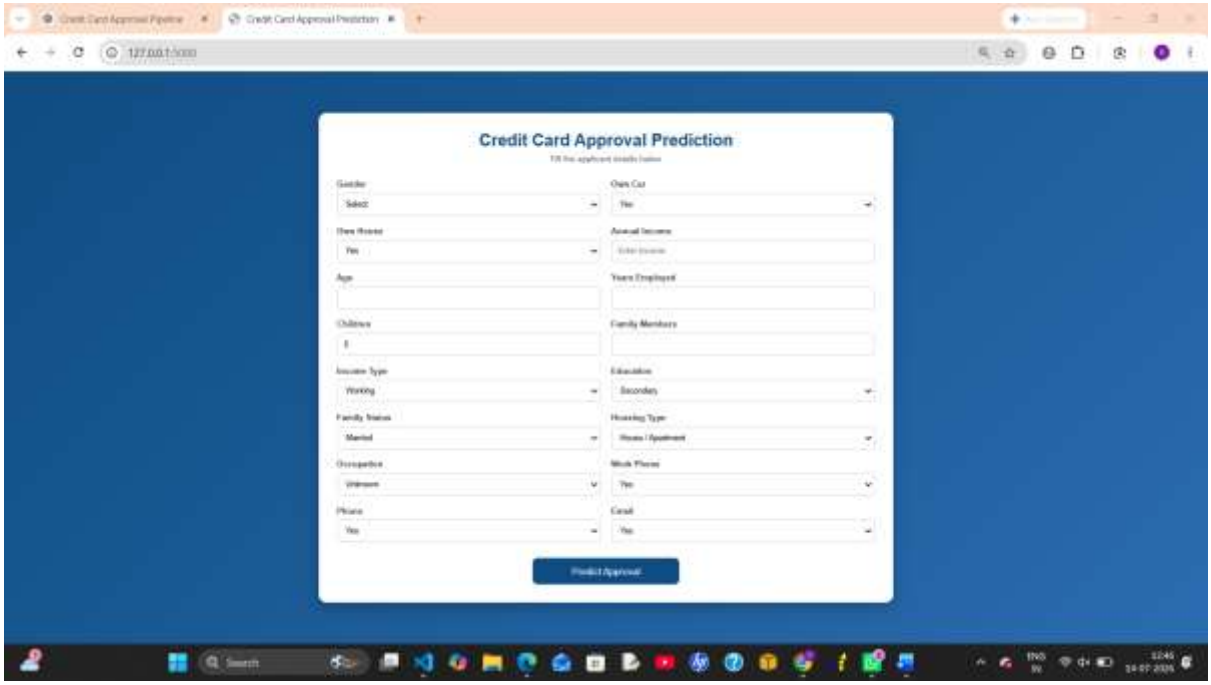
The following aspects were verified:

- Application startup time.
- Prediction response time.
- Form submission.
- Page loading.
- Model loading.
- Overall application stability.

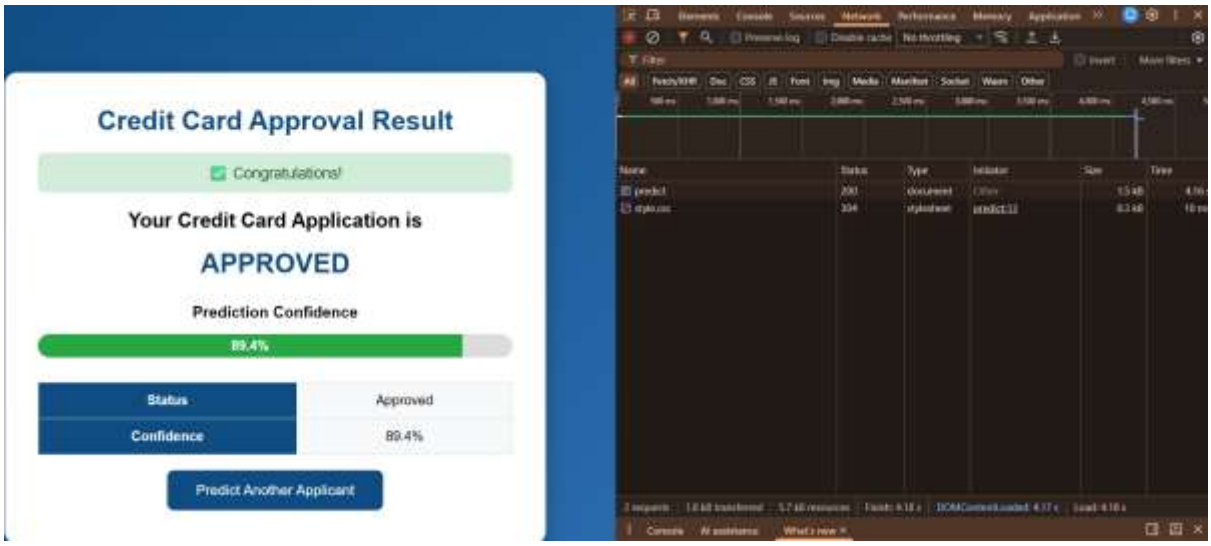
The prediction result was generated within a few seconds, providing a smooth user experience.

Home Page

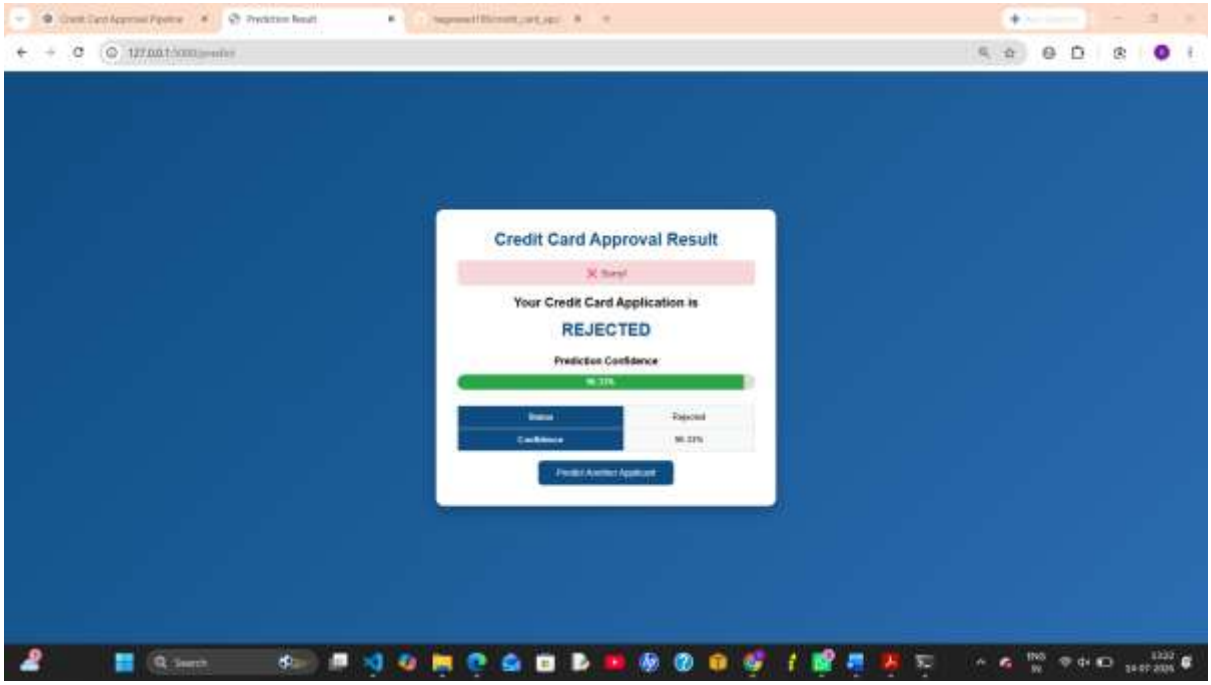
Applicant Details Form



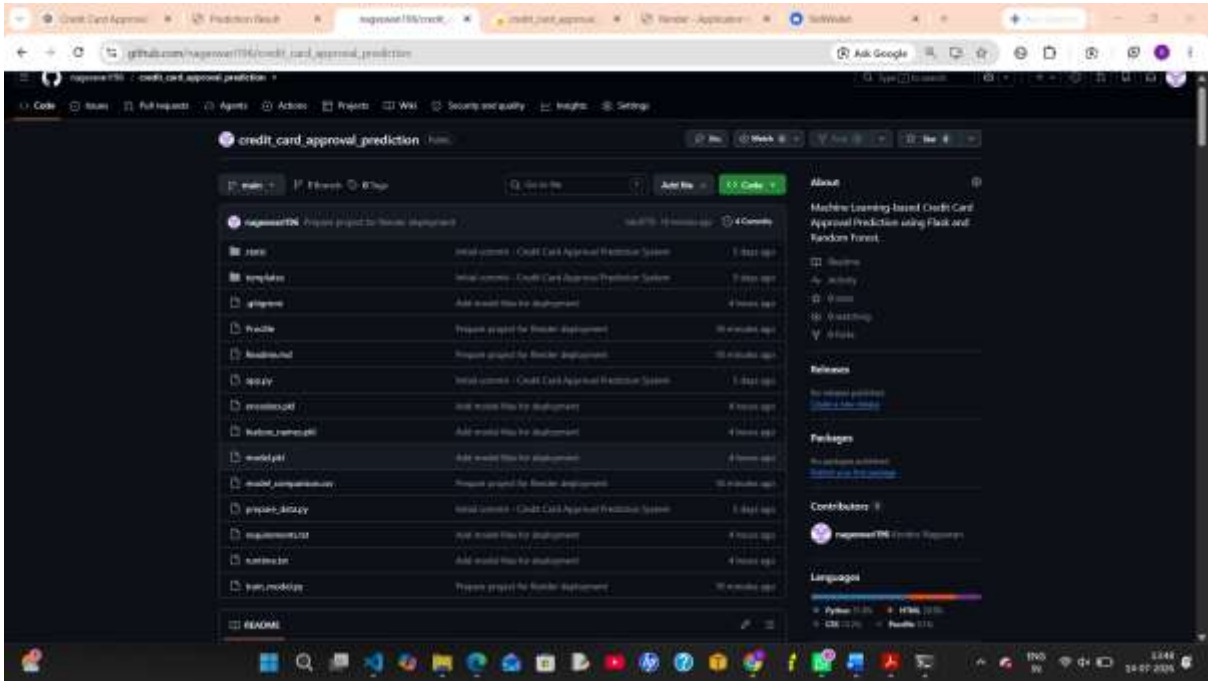
Approved Prediction Result



Rejected Prediction Result



GitHub Repository



Project Results

The Credit Card Approval Prediction System successfully predicts whether an applicant's credit card request is likely to be approved or rejected using machine learning techniques.

The project performs the following tasks:

- Collects applicant details.
- Cleans and preprocesses the data.
- Performs feature engineering.
- Encodes categorical values.
- Applies the trained Random Forest model.
- Predicts the approval status.
- Displays the prediction confidence.
- Provides a simple web interface for users.

The system demonstrated good prediction performance and successfully integrated machine learning with a web application.

Conclusion

The **Credit Card Approval Prediction System** successfully demonstrates the application of Machine Learning in automating the credit card approval process.

The project began with collecting and preprocessing applicant data, followed by feature engineering and label encoding to prepare the dataset for model training. Multiple machine learning algorithms, including Logistic Regression, Decision Tree, Random Forest, and XGBoost, were trained and evaluated. Based on the evaluation results, the Random Forest model was selected as the final prediction model due to its reliable performance and balanced accuracy.

The trained model was integrated into a Flask-based web application that enables users to enter applicant information and receive instant credit card approval predictions along with a confidence score. The application was tested using different applicant details and handled predictions accurately while providing a simple and user-friendly interface.

Overall, the project demonstrates how Machine Learning and Web Development can be combined to create an intelligent decision-support system for the banking sector. It reduces manual effort, improves consistency in decision-making, and provides faster prediction results.

Future Scope

The project can be further enhanced by implementing the following features:

- Integration with real-time banking databases.
- User authentication and login system.
- Explainable AI techniques such as SHAP or LIME to explain predictions.
- Cloud-based database integration using MySQL or PostgreSQL.
- REST API development for mobile and third-party applications.
- Advanced machine learning and deep learning models for improved accuracy.
- Real-time credit score integration.
- PDF report generation for applicants.
- Dashboard for bank administrators to monitor prediction statistics.

These improvements would make the system more scalable, secure, and suitable for real-world banking applications.

Acknowledgement

We express our sincere gratitude to our project guide, faculty members, and the Department of Computer Science and Engineering for their continuous guidance and support throughout the development of this project.We also thank the developers of open-source libraries such as Flask, Scikit-learn, Pandas, NumPy, XGBoost, and Matplotlib for providing powerful tools that helped us successfully complete this project.

Finally, we thank our institution and teammates for their encouragement and cooperation during the project development.